AWS CLOUD INFRASTRUCTURE PROJECT

# Private EC2 Web Servers with Bastion Host & Application Load Balancer

A hands-on, multi-Availability-Zone AWS networking and web hosting implementation

| PROJECT SUMMARY | DETAILS |
| --- | --- |
| Region | US East (Ohio) - us-east-2 |
| Core services | Amazon VPC, EC2, Security Groups, Elastic Load Balancing |
| Application | Static HTML served by Python HTTP server |
| Outcome | Two private targets reported healthy; public HTTP ALB endpoint served website |
| Documentation basis | Observed AWS console screenshots and terminal sessions, November 6, 2026 |

Portfolio note: This report records the tested lab environment. It does not claim production hardening, HTTPS, or confirmed deployment of every optional service in the AWS reference diagram.

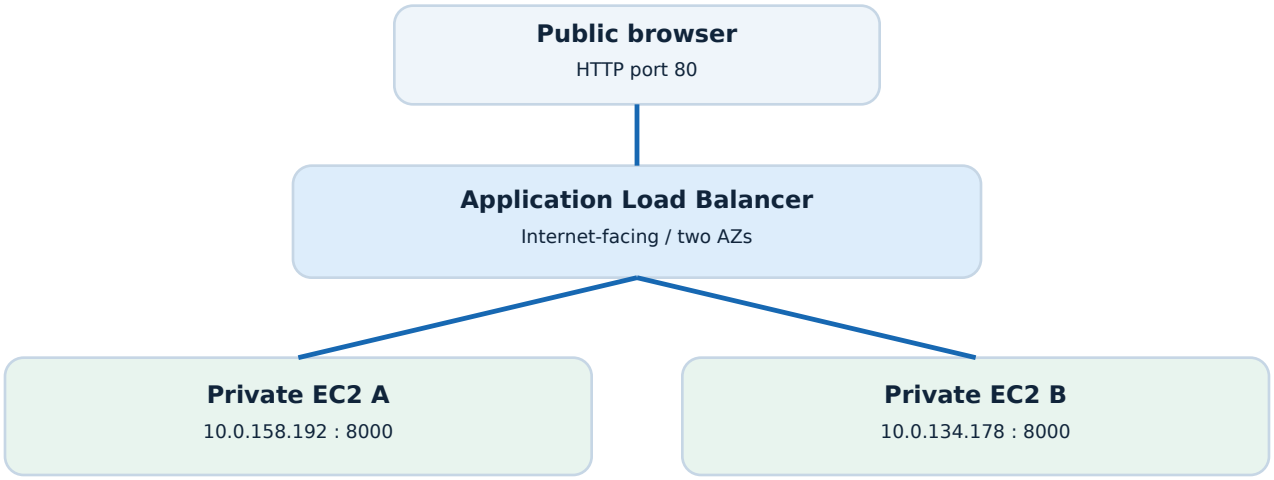
# 1. Executive Summary

This project demonstrates how to host a simple web application on private Amazon EC2 instances while exposing the application through an internet-facing Application Load Balancer (ALB). Administrative SSH access to the private instances was established through a bastion host. Two EC2 instances in separate Availability Zones were registered in a target group on port 8000 and were ultimately shown as healthy.

The application was accessed successfully using the ALB DNS name with an explicit http:// scheme. Earlier attempts produced connection errors, which were resolved during configuration and testing.

# 2. Architecture Overview

Browser / Internet -> Internet-facing Application Load Balancer (HTTP :80) -> Target Group (HTTP :8000) -> Private EC2 instances in two Availability Zones. Separately, administrator -> bastion host (public IP, SSH :22) -> private EC2 SSH :22.




The bastion host is used for management access and is not on the application HTTP request path. The ALB and its targets were associated with the project VPC.

# 3. Observed Environment

| RESOURCE | OBSERVED CONFIGURATION |
|---|---|
| AWS Region | us-east-2 (Ohio) |
| Project VPC | vpc-033b351406c85aefc |
| Private EC2 A | 10.0.158.192; us-east-2b; Ubuntu 26.04; t3.micro |
| Private EC2 B | 10.0.134.178; us-east-2a; Ubuntu 26.04; t3.micro |
| Bastion public IP used in SSH | 18.118.128.241 |
| ALB | my-load-balancer; Application; internet-facing; Active |
| ALB listener | HTTP :80 forwarding to my-target-group |
| Target group | Instance targets; HTTP :8000; two registered and healthy |
| Website endpoint | http://my-load-balancer-990365910.us-east-2.elb.amazonaws.com/ |

# 4. Implementation Workflow

## Step 1 - Network and instances

• Created or configured the project VPC and subnets across two Availability Zones.

• Used a public bastion host to access private EC2 instances that had no public IPv4 addresses.

• Launched two Ubuntu EC2 web servers in private subnets and configured security groups for SSH and application traffic.

## Step 2 - SSH through bastion host

SSH authentication initially failed with Permission denied (publickey) because the correct PEM identity was not consistently supplied to the SSH hops. A Windows local key path was also mistakenly used inside the Linux shell. Using the proper key and jump-host configuration established access to both private instances.

Representative command structure (run from Windows PowerShell; use the actual key path):

```
ssh -i "$env:USERPROFILE\Downloads\aws.pem" -J ubuntu@18.118.128.241 ubuntu@10.0.134.178
```

For separate jump-host and destination key handling, SSH ProxyCommand or a configured SSH client can be used. Keep private keys on the local workstation; do not upload PEM files to GitHub.

## Step 3 - Create a test web page

Created index.html on the private EC2 web server. The HTML contained a welcome heading, a private EC2 web-server heading, and a short description of the bastion and Python hosting setup.

```
ls -l index.html
```

### Step 4 - Start Python web server

Started the Python HTTP server from the directory containing index.html. For ALB access, the service must listen on the instance network interface, not exclusively on loopback 127.0.0.1.

```
python3 -m http.server 8000 --bind 0.0.0.0
```

A prior --bind 127.0.0.1 configuration allowed SSH-local port forwarding but did not allow the ALB to reach the server through its private IP. The test server was later made reachable for ALB health checks. The Python server is appropriate for a learning demo, not production hosting.

### Step 5 - Configure load balancing

• Created the internet-facing Application Load Balancer named my-load-balancer.

• Configured an HTTP port 80 listener to forward to my-target-group.

• Registered both private EC2 instances as targets on HTTP port 8000.

• Checked target group status and observed both targets Healthy.

### Step 6 - Validate application

Opened the ALB DNS name with an explicit HTTP scheme. The browser displayed the application heading Welcome to My First AWS Project! and the page content served from the private web tier.

## 5. Troubleshooting and Lessons Learned

| SYMPTOM | CAUSE / RESOLUTION |
|---|---|
| SSH: Permission denied (publickey) | Wrong or missing private-key identity on one SSH hop; supply correct key to both hops. |
| Linux shell could not find Windows key path | Windows environment variables and file paths are not available inside the remote Ubuntu shell. |
| Browser localhost:8000 refused connection | Python server stopped, or SSH local forwarding was not active. Keep server and tunnel running. |
| ALB target Unhealthy | One instance was not yet reachable/responding on port 8000; verify binding, process, health check, and security-group rules. |
| ALB DNS link timed out | An explicit http:// URL later worked. Check browser scheme, ALB inbound HTTP :80, routing and target health if timeout recurs. |

## 6. Validation Evidence

| TEST | OBSERVED RESULT |
|---|---|
| Bastion access | Successful Ubuntu SSH login. |
| Private instance access | Successful SSH logins to 10.0.158.192 and 10.0.134.178. |
| Web content | index.html created and listed on private instance. |
| Python server | Serving HTTP on port 8000; successful local request shown. |
| Target group health | Both registered targets reported Healthy in AWS console. |
| Public application | HTTP ALB URL displayed project website in browser. |

## 7. Security and Production Considerations

• For a lab, HTTP on port 80 works; for production, configure HTTPS with an ACM certificate and a port 443 listener.

• Allow inbound port 8000 on EC2 only from the ALB security group; do not expose private instances directly to the internet.

• Restrict bastion SSH access to trusted source IPs; consider AWS Systems Manager Session Manager as an alternative.

• Replace Python http.server with a supported production web server or application stack for real workloads.

• Use IAM least privilege, patching, monitoring, logging, and budget alerts.

• The project demonstrated working load-balanced delivery, but equal traffic distribution and automatic scaling were not separately measured.

## 8. Optional Architecture Components - Not Fully Verified

| COMPONENT | STATUS / WHY IT MATTERS |
|---|---|
| NAT gateways | Not verified. Enable outbound internet access for private workloads if needed; ongoing cost applies. |
| S3 gateway endpoint | Not verified. Can provide private VPC route access to S3. |
| Auto Scaling Group | AWS console referenced an ASG name, but its effective configuration, instance attachment, and scaling behavior were not verified. |
| Internet gateway / route tables | ALB public access succeeded; complete route-table design was not separately audited. |
| HTTPS / TLS | Not configured in the demonstrated HTTP-only browser test. |

## 9. Conclusion

This hands-on AWS project successfully demonstrated hosting a web application on two private EC2 instances across separate Availability Zones. A bastion host provided administrative SSH access, while an internet-facing Application Load Balancer routed HTTP requests to the instances through a target group.

Both registered targets were confirmed healthy, and the application was successfully accessed through the load balancer's HTTP endpoint. Troubleshooting SSH authentication, server binding, target health checks, and connectivity provided practical experience with cloud networking and deployment.

The result is a working lab that demonstrates core AWS VPC, EC2, security group, and load balancing concepts. It is a learning implementation rather than a fully hardened production deployment.